

CENG 589 Project Report

Ayşe Aysu Cengiz
2580371

1 About Project

This project aims to achieve a mesh interpolation scheme, where there will be no self-intersections in intermediate steps of the interpolation. For this purpose, surfmorph [1] and Instant Self-Intersection Repair for 3D Meshes [2] are used. Detailed information about the repositories are in 2. Github repository: aysucengiz/surfmorph_self_intersection_repair

2 Tools

2.1 Surfmorph

Surfmorph is an implementation of As-Rigid-As-Possible [3] surface morphing. It is a header only repository which also includes an application that shows the in-between steps of the interpolation as an animation. For this project, this application was modified.

The repository only requires eigen [4]. The interface includes a SurfaceMorph class with main methods addPose and interpolatePoseAt. The class is initialized with a minimum of two poses and each pose is given a timestep based on their index. This means if we call interpolatePoseAt(2.2) we will get the interpolation of pose[2] and pose[3] with 0.2 factor.

The app additionally requires OpenMesh [5], Qt5 [6] and libQGLViewer [7]. It has an animator class that includes generateAnimationFrames, which calls generateFramePose. These are the functions that are primarily important for the project.

2.2 Instant Self-Intersection Repair for 3D Meshes

This repository is used as a black box where it is simply called with repair_factory.py. The program makes use of NVIDIA CUDA to speed up the process. It creates multiple repaired meshes and labels the best one.

2.3 Meshes and Blender

I searched for example meshes yet the meshes from our previous assignments were good enough. I also searched for meshes and found a github repository with common 3d meshes by alecjacobson [8].

I used Blender's Pose tool to modify meshes in order to create interpolations that would intersect at least one step. Some poses do seem like the living beings have broken bones but it was my first time using blender and that was not the main idea.

3 Algorithms

3.1 Independent Repair

This was the very first and easiest solution that came to mind. The idea was to do the interpolation just the same and repair any intersecting intermediate meshes.

3.2 Incremental Repair

Another way I tried was the incremental repair.

1. Call the first mesh M_0 and the second M_N .
2. At n^{th} frame, apply interpolation by referencing the $(n - 1)^{th}$ frame:

$$M_n = M_{n-1} \frac{N - n - 1}{N - n} + M_N \frac{1}{N - n}$$

3. If M_n self-intersects, repair it.
4. Increment n and Continue until $n = N$.

This has the risk of drifting away, so independent one can be more robust.

4 Methodology

4.1 Instant Self-Intersection Repair for 3D Meshes

The main struggle with this repository was that it required nvidia cuda which wouldn't work with my computer. To fix this issue, I planned a pipeline to connect to our university's lab computers and run this repository via ssh and wrote a script with the assistance of generative ai since this is more of a hardware issue than a mesh interpolation issue. Even with that the process took longer than expected.

I also tried different self-intersection repair repositories yet settled on this one since its speed was redeemable and the repair itself was a more robust. The connection details are given at 4.2. Another issue was that this repository is big, and its requirements take quite the memory which is more than the quota my account has. I hardly fit the virtual environment into the 'tmp' folder which usually has 8.8GB free space. That is why the project is not entirely done on the lab computers.

However, as the name suggests, contents of 'tmp' are volatile and I would need to create the virtual environment together with requirements over and over again. Moreover, just installing requirements do not work since by default some libraries come with conflicting versions and if I try to download the proper version even the tmp's quota is filled. For this purpose, I compiled my commands to a bash script file to speed up this process.

The script:

1. Deletes old tmp files.
2. Prints the capacity.
3. Initializes venv under /tmp/e2580371/imrepair_cuda11 and activates it.
4. Installs torch 1.13.1 with cuda 11.6
5. Installs requirements for the original repository: numpy<2, scipy, PyYAML, potpourri3d, cholepsy, setuptools 69.5.1. (Certain versions are forced for compatibility)
6. Installs requirements for the torch-mesh-isect with g++10 and then runs setup.py install.
7. Finishes the process by printing final checks.

4.2 Connection Script

The script initially removes my predetermined cache files under my home directory (tmp is handled by the previously mentioned setup script) to allow multiple runs without the necessity to ssh myself and remove them.

It then opens the ssh and sftp connections where the credentials and information is given via the Repair class that calls it. The class gets the information from a ".env" file.

After the connections are open, the script prints "obj>" and waits for an object path. When that path is given it first puts the object to the lab computer via sftp and runs the repair program. The console output is redirected to our original output.

When the program is done, we get the file that is labeled as best via sftp.

Until q is sent, the program stays in a loop that waits for object paths to repair.

4.3 Surfmorph Modification

After handling every issue with mesh repair, I moved on to Surfmorph. The idea was to modify the qt app that shows the interpolation steps in between as 10 steps by calling my connection script in-between.

4.3.1 Setup Issues

Since I originally used wsl 2, I faced the issue of VPN not reaching the wsl. Instead of simply mirroring the network I tried to compile surfmorph in windows. Definitely not recommended as it took hours of trying to fixing mismatched versions (Qt 5 is archived and the ones I found would not work) and deleting/downloading MinGW and MSVC versions of the repositories. In the end just by mirroring the network of the wsl I made it work with some minor fixes.

Another issue was the app's parallel option. Qt would keep popping up warnings and then having a segmentation fault when I incorporated my script into the app. So I deleted the parallelization parts. There is normally a flag to turn it on and off but that would be meaningless as I was never going to use it that way. This made the process slower.

4.3.2 Repair Class

I defined a repair class that would open the ssh and sftp initially via the script and send requests as the program continued. The class would then close the connections as it was being destructed. The class reads from the env file and determines the credentials. The main idea is to use it as an interface to the script.

4.4 Algorithm Implementation

To implement the algorithms I modified the previously mentioned surfmorph app functions.

I also added a parameter "r" that lets the user choose if they want the repair or not. If repair is chosen, repairFramePoseIfNeeded function is called. There are two versions of generateFramePose, one for incremental one for independent repair. The independent one is simple, it just runs repair if there is a problem. The incremental one recomputes the initial pose and continues from there.

5 Results

The results were a lot more satisfactory from the beginning. The only problem is that if a part of the mesh is going "through" the mesh itself which results in a jumping motion. I initially thought that problem was solvable with the tools at hand. Yet it requires a bigger scale mesh manipulation, one that possibly finds a safe route for that piece of mesh because the repair script simply chooses the easiest way out for that intersection and usually just pushes it back/front, not to the side. For now, the solution for is for the user to present an intermediate mesh.

-
- **Independent Repair:** I had expected this to have more problems yet it worked flawlessly.
 - **Incremental Repair:** This one was a bit more problematic which is not visible at first sight. It is a bit more "shaky".

Youtube video for both: <https://youtu.be/72IH-caJXMc>

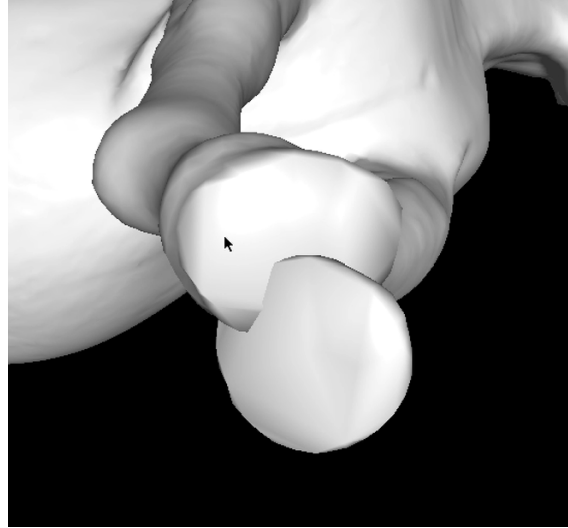


Figure 1: Intersection before the repair.

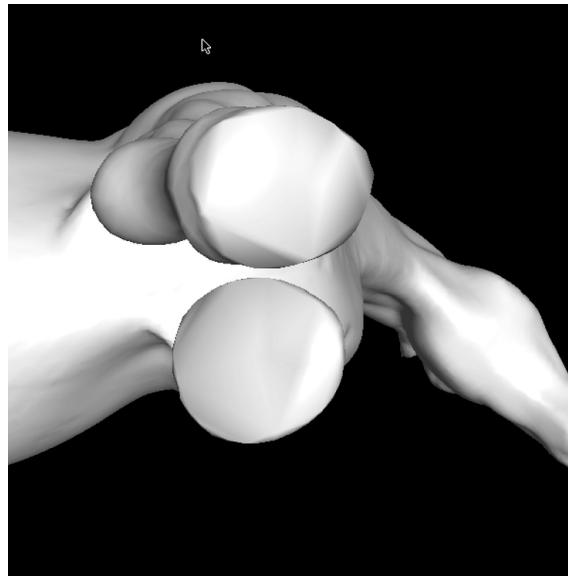


Figure 2: After repair: No intersection

References

- [1] C. F. Javid, *Surfmorph*, GitHub repository. Accessed: 2026-06-25, 2026. [Online]. Available: <https://github.com/javidcf/surfmorph>.

- [2] W. Jang, Y. Jung, G. Lee, and S. Lee, “Instant self-intersection repair for 3d meshes”, *ACM Transactions on Graphics (Proceedings of ACM SIGGRAPH)*, vol. 44, no. 4, 2025.
- [3] O. Sorkine and M. Alexa, “As-rigid-as-possible surface modeling”, in *Proceedings of the Fifth Eurographics Symposium on Geometry Processing*, A. Belyaev and M. Garland, Eds., The Eurographics Association, 2007, pp. 109–116. doi: 10.2312/SGP/SGP07/109-116.
- [4] G. Guennebaud, B. Jacob, et al., *Eigen v3*, 2010. [Online]. Available: <https://eigen.tuxfamily.org>.
- [5] M. Botsch, S. Steinberg, S. Bischoff, and L. Kobbelt, “Openmesh – a generic and efficient polygon mesh data structure”, in *OpenSG Symposium*, 2002.
- [6] Qt Group, *Qt: Cross-platform software development framework*, 2024. [Online]. Available: <https://www.qt.io>.
- [7] N. Robert, *Libqglviewer: A c++ framework for 3d opengl visualization*, <https://github.com/GillesDebunne/libQGLViewer>, 2005.
- [8] A. Jacobson, *Common 3d test models*, <https://github.com/alecjacobson/common-3d-test-models>, GitHub repository, accessed 2026-06-25, 2020.